

Shruti Pandey - 25MML0020

## VIDEO DETECTION

---

# EDGE INTELLIGENCE - VIDEO DETECTION

Shruti Pandey 25MML0020

Importing Libraries

Restore Up

[1]:

```
# Import required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

Starting live video and checking for security

[2]:

```
# Opening webcam (0 = default camera)
cap = cv2.VideoCapture(0)

# Security check to confirm camera access
if not cap.isOpened():
    print("Camera access failed")
else:
    print("Camera started successfully")
```

Camera started successfully

## Displaying video properties

[3]:

```
# Getting video properties
fps = cap.get(cv2.CAP_PROP_FPS)
width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)

print("FPS:", fps)
print("Width:", width)
print("Height:", height)
```

Restore Up

```
FPS: 30.0
Width: 640.0
Height: 480.0
```

## Counting frames while capturing

[4]:

```
frame_count = 0

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame_count += 1
    cv2.imshow("Live Video", frame)

    if cv2.waitKey(1) == ord('q'):
        break

print("Total frames captured:", frame_count)

cap.release()
cv2.destroyAllWindows()
```

Restore Up

```
Total frames captured: 526
```

Capturing one frame and displaying it

[7]:

```
cap = cv2.VideoCapture(0)

ret, frame = cap.read()

print("Frame shape:", frame.shape)

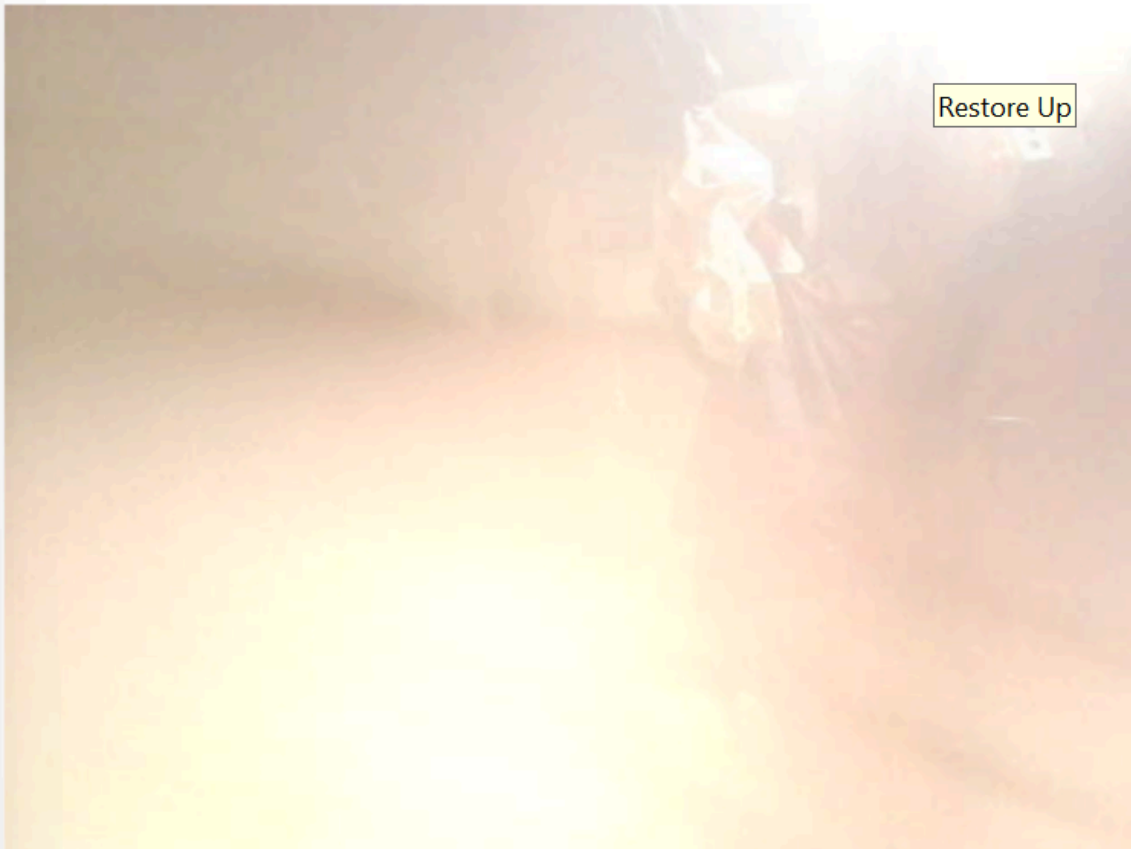
plt.imshow(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
plt.title("Captured Frame")
plt.axis("off")

cap.release()
```

Restore Up

Frame shape: (480, 640, 3)

Captured Frame



Restore Up

Converting to grayscale and printing matrix

[8]:

Restore Up

```
# Converting frame to grayscale
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

plt.imshow(gray, cmap='gray')
plt.title("Grayscale Frame")
plt.axis("off")

# Print grayscale matrix
print("Grayscale matrix:")
print(gray)
```

Grayscale matrix:

```
[[171 171 171 ... 255 255 255]
 [171 171 171 ... 255 255 255]
 [171 171 171 ... 255 255 255]
 ...
 [243 243 243 ... 236 236 236]
 [243 243 243 ... 237 237 237]
 [243 243 243 ... 237 237 237]]
```

Grayscale Frame



Restore Up

Printing pixel values of both colored and gray scales

[9]:

```
# Selecting a pixel location
x = 100
y = 100

# Colored pixel value
print("RGB pixel value:", frame[y, x])

# Grayscale pixel value
print("Gray pixel value:", gray[y, x])
```

Restore Up

RGB pixel value: [165 187 205]

Gray pixel value: 190

Saving every 100th frame

[10]:

```
cap = cv2.VideoCapture(0)

count = 0
saved = 0

while True:
    ret, frame = cap.read()
    if not ret:
        break

    if count % 100 == 0:
        filename = f"frame_{saved}.jpg"
        cv2.imwrite(filename, frame)
        saved += 1

    count += 1

    cv2.imshow("Video", frame)
    if cv2.waitKey(1) == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Restore Up

Showing frame dimensions

[11]:

```
print("Frame dimensions:", frame.shape)
```

Frame dimensions: (480, 640, 3)

Restore Up

Drawing various shapes using CV2

[13]:

```
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Rectangle
    cv2.rectangle(frame, (100,100), (300,300), (0,255,0), 3)

    # Circle
    cv2.circle(frame, (200,200), 50, (255,0,0), 3)

    # Text
    cv2.putText(frame, "OpenCV", (50,50),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255), 2)

    cv2.imshow("Shapes", frame)

    if cv2.waitKey(1) == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Restore Up



### Edge Detection using Canny

[14]:

```
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

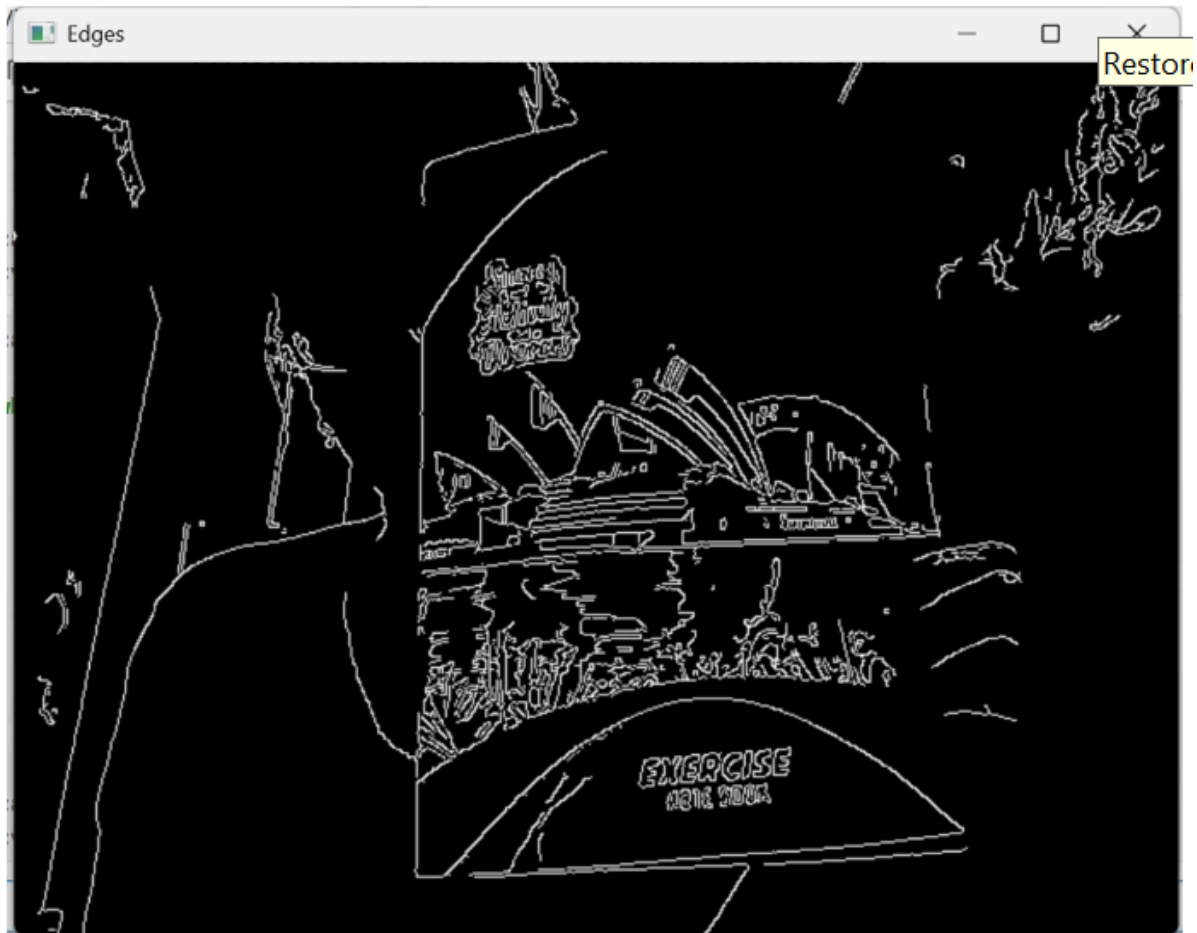
    edges = cv2.Canny(frame, 100, 200)

    cv2.imshow("Edges", edges)

    if cv2.waitKey(1) == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Restore Up





Negative of video

[17]:

```
cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("Error: Unable to access camera")
    exit()

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Increasing contrast for sharper appearance
    enhanced = cv2.convertScaleAbs(frame, alpha=1.5, beta=0)

    # Creating negative image
    negative = 255 - enhanced

    cv2.imshow("Sharp Negative Video", negative)

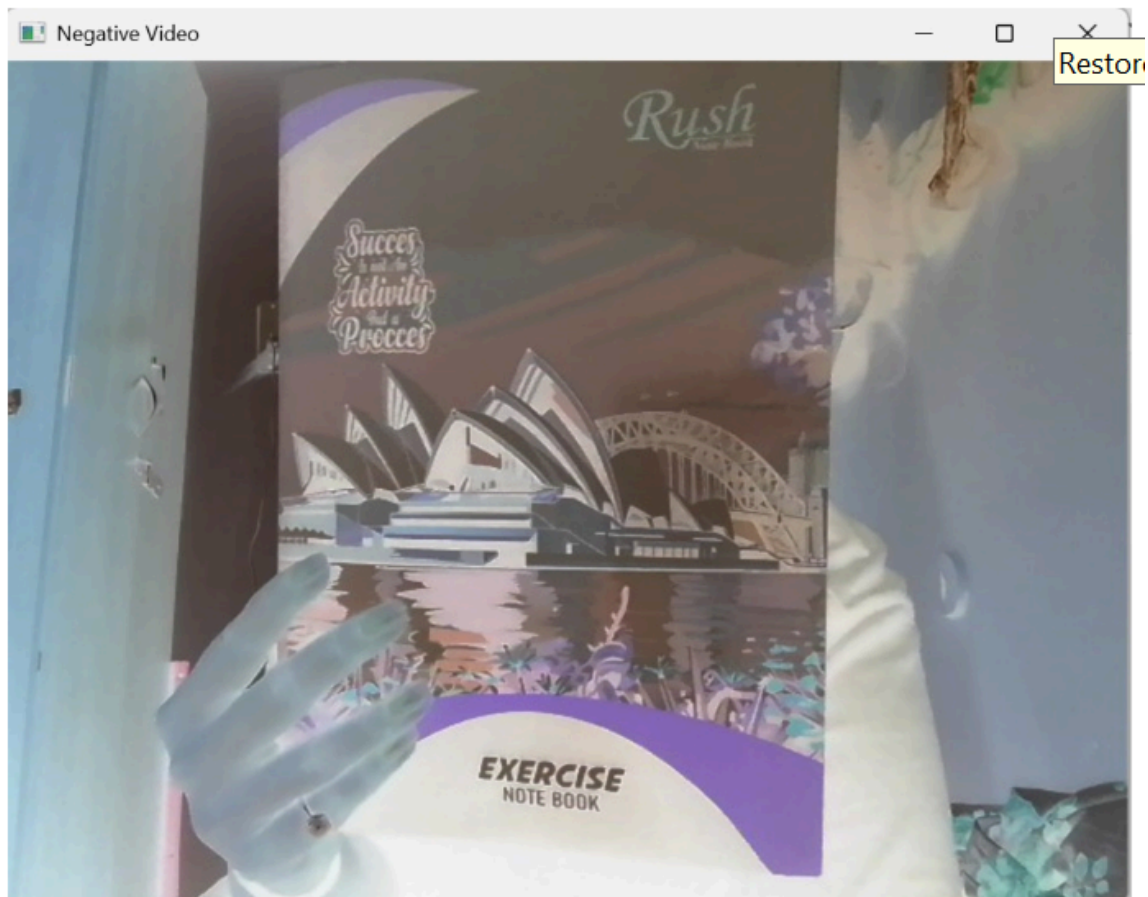
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Restore Up



Negative of image - 1



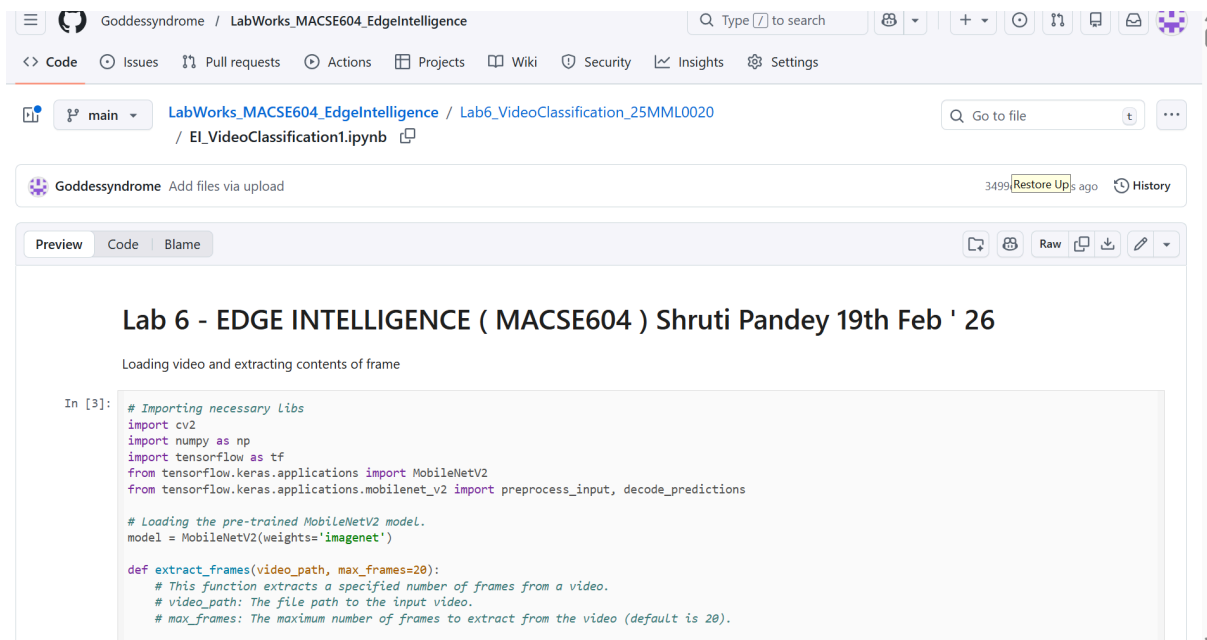
Negative image - 2 ( improved )



## VIDEO CLASSIFICATION - ON GITHUB (

[https://github.com/Goddessyndrome/LabWorks\\_MACSE604\\_EdgeIntelligence/blob/main/Lab6\\_VideoClassification\\_25MML0020/EI\\_VideoClassification1.ipynb](https://github.com/Goddessyndrome/LabWorks_MACSE604_EdgeIntelligence/blob/main/Lab6_VideoClassification_25MML0020/EI_VideoClassification1.ipynb)

)



## Discussion on Today's Discussion - Java Backend for Number Plate Detection

This project builds a simple AI-based system that detects vehicle number plates from images or video frames and shows the result to the user. The main goal is to understand how images move through a complete AI pipeline from input to output using computer vision and backend processing.

First, the system receives an input image or video. The input can come from a camera, a stored video file, or an uploaded image. This input is sent to the backend system through an API or application interface. The backend server acts as the main controller that manages all operations in the system.

Once the image is received, the system performs pre-processing. In this stage the image is cleaned and prepared so that the AI model can process it properly. Typical preprocessing steps include resizing the image, normalizing pixel values, and converting formats if necessary.

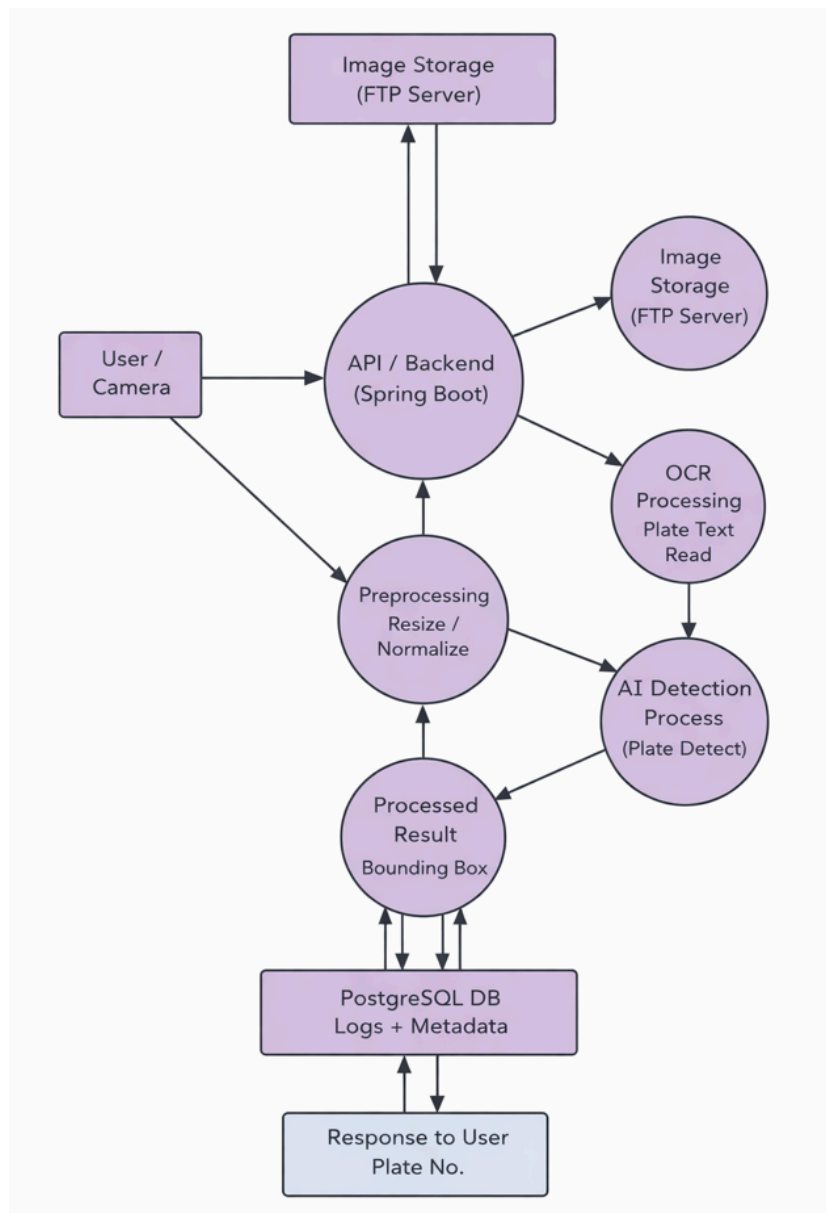
After preprocessing, the image enters the AI detection stage. A computer vision model analyzes the image to locate the vehicle number plate. The model identifies the region of the image where the plate is present and draws a bounding box around it.

Next, the system performs post-processing. In this step the detected plate region can be enhanced, cropped, or passed to an OCR module to extract the actual text written on the number plate.

The system also stores important information in a database. The database records details such as uploaded file name, storage location, processing time, and detected results. This helps in tracking and auditing system operations.

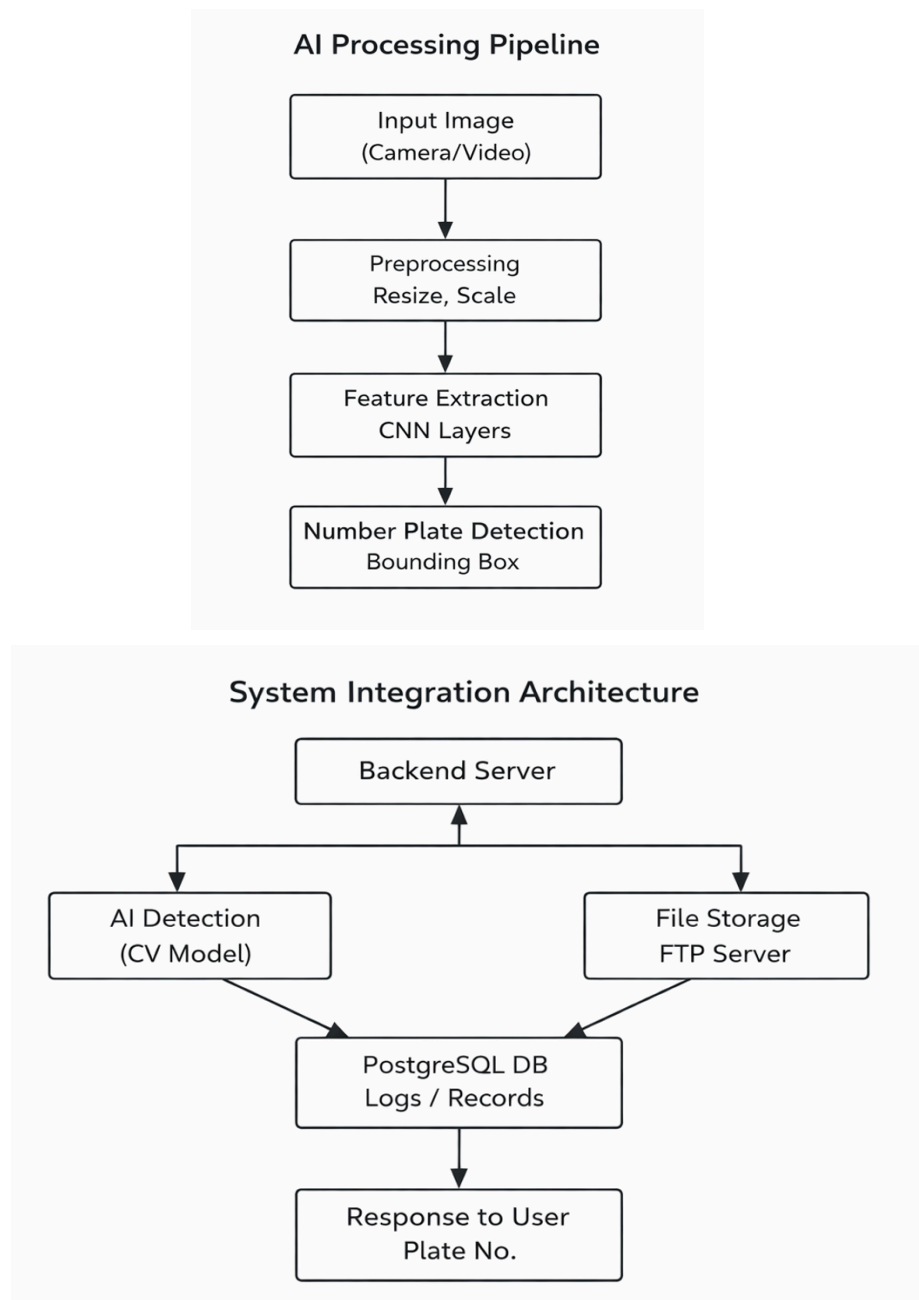
Finally, the processed output is returned to the user. The user receives an image with the detected number plate highlighted or the extracted plate number as text.

### System Architecture Diagram :



In the above flow diagram , the system receives a vehicle image from a user or camera and sends it to the backend through an API. The image is preprocessed and analyzed by an AI model to detect the number plate. OCR reads the plate text. Results are stored in a database, and the detected plate information is returned to the user.

## AI Pipeline and system integration flowcharts :



The AI pipeline is used to process images. An input image from a camera is preprocessed, features are extracted using CNN layers, and the system detects the number plate using a bounding box. OCR then reads the plate text and produces the final annotated output.

The second diagram is of the system integration where the backend server connects the AI detection model and file storage. Results and logs are stored in a PostgreSQL database, and the processed plate information is returned to the user.

## **How Input - Output works :**

**User Uploads Image** – The user captures or uploads a vehicle image or video frame into the system through a camera, web interface, or mobile application

**Backend Receives Request** – The backend server receives the uploaded image through an API and prepares it for further processing.

**AI Model Detects Plate** – A computer vision model analyzes the image to locate the number plate region within the vehicle.

**Bounding Box Added** – The system marks the detected number plate area by drawing a bounding box around it.

**Plate Number Extracted** – OCR technology reads the characters on the number plate and converts them into text.

**Result Sent Back to User** – The processed result, including the detected plate and extracted number, is returned to the user.

## **Role of RDBMS :**

It stores and organizes data in structured tables with rows and columns. In this system, it records uploaded image details, processing results, and detected number plates. It helps manage logs, maintain relationships between data, and allows efficient querying, retrieval, and tracking of system operations.

## **Applications of the System:**

- Smart traffic monitoring
- Automatic toll collection
- Parking automation systems
- Smart city surveillance
- Law enforcement vehicle tracking
- Entry control systems in buildings
- Fleet monitoring systems